

```
1 use ciborium::into_writer;
2 use ciborium::value::Value;
3 use clap::{ArgAction, Parser};
4 use color_eyre::{
5     Result,
6     eyre::{Context, eyre},
7 };
8 use log::{LevelFilter, debug, warn};
9 use std::{collections::HashMap, fs::File, io::BufWriter, path::PathBuf};
10 use std::{
11     fs,
12     io::{Write, stdout},
13 };
14 use std::{path::Path, result::Result::Ok};
15
16 /// A command line utility that allows you to "pack" a directory and its
17 /// contents
18 /// into a CBOR (see https://cbor.io/) representation.
19 #[derive(Parser, Debug)]
20 #[command(
21     author,
22     version,
23 )]
24 struct Args {
25     /// Only include files up to a certain size
26     ///
27     /// You may specify a size using equals:
28     /// --max-size => defaults to 40KiB
29     /// --max-size=1024 => limit of 1024 bytes
30     /// --max-size=3MB => limit of 3MB
31     #[arg(
32         short,
33         long,
34         require_equals=true,
35         default_missing_value="40KiB",
36         value_parser=parse_byte_size,
37         num_args=0..=1,
38         verbatim_doc_comment
39     )]
40     max_size: Option<u64>,
41     /// Only include files that are valid UTF-8 text
42     ///
43     /// On occasion this may include non-text files if
44     /// their contents happen to be parsable as UTF-8
45     #[arg(short, long)]
```

```

45     text_only: bool,
46     #[arg(short, long)]
47     /// Changes leaf content to (filename, content) pairs
48     ///
49     /// This is useful if you intend to display the filename
50     /// along the file contents
51     include_file_name: bool,
52     /// Increase verbosity (repeat for more: -v info, -vv debug, -vvv trace)
53     #[arg(short, long, action = ArgAction::Count)]
54     verbose: u8,
55     /// Path to the directory to pack
56     path: PathBuf,
57 }
58
59 const CLEAR_LINE: &str = "\r\x1B[K";
60
61 fn main() -> Result<()> {
62     color_eyre::install()?;
63
64     // Check if this is a directory
65     let args = Args::parse();
66     debug!("Received arguments: {args:?}");
67
68     // Set logger verbosity
69     env_logger::Builder::new()
70         .filter_level(match args.verbose {
71             0 => LevelFilter::Warn,
72             1 => LevelFilter::Info,
73             2 => LevelFilter::Debug,
74             _ => LevelFilter::Trace,
75         })
76         .init();
77
78     if !args.path.is_dir() {
79         return Err(eyre!(format!("{0} is not a directory",
80             args.path.display())));
81     }
82
83     // Build the file tree
84     let tree = build_file_tree(
85         Path::new(&args.path),
86         args.max_size,
87         args.text_only,
88         args.include_file_name,
89     )?;
90     print!("{CLEAR_LINE}");

```

```

90
91     // Determine the output file name
92     let file_name = format!(
93         "{}.cbor",
94         args.path
95             .canonicalize()
96             .ok()
97             .and_then(|path| path
98                 .file_name()
99                 .map(|name| name.to_string_lossy().into_owned()))
100         .unwrap_or_else(|| {
101             warn!(
102                 "Could not canonicalize serialized directory path.\n\
103                 Using fallback name..."
104             );
105             "arbor".to_string()
106         })
107     );
108
109     let file = File::create(&file_name).wrap_err("Couldn't create output
110     file")?;
111     let mut buffered = BufWriter::new(file);
112     into_writer(&tree, &mut buffered).unwrap();
113     println!("Wrote cbor file to {}", file_name);
114     Ok(())
115 }
116
117 fn build_file_tree(
118     root: &Path,
119     max_size: Option<u64>,
120     text_only: bool,
121     include_file_path: bool,
122 ) -> Result<Value> {
123     let mut map: HashMap<String, Value> = HashMap::new();
124
125     stdout().flush()?;
126
127     let read_dir =
128         fs::read_dir(root).wrap_err(format!("Couldn't read directory {}",
129         root.display()))?;
130
131     for entry in read_dir.flatten() {
132         let entry_path = entry.path();
133         let name = entry.file_name().to_string_lossy().to_string();
134
135         let value = if entry_path.is_file() {

```

```

134         // Turn this file into a value
135         read_file_to_value(&entry_path, max_size, text_only,
136             include_file_path)?
137     } else if entry_path.is_dir() {
138         // Keep building
139         build_file_tree(&entry_path, max_size, text_only,
140             include_file_path)?
141     } else {
142         continue;
143     };
144     map.insert(name, value);
145 }
146 Value::serialized(&map).wrap_err(format!("Couldn't serialize to CBOR
147 {map:?}"))
148 }
149 /// Read the file at `entry_path` and return a fitting Value based on
150 /// the given options
151 fn read_file_to_value(
152     entry_path: &PathBuf,
153     max_size: Option<u64>,
154     text_only: bool,
155     with_file_path: bool,
156 ) -> Result<Value, color_eyre::eyre::Error> {
157     // There is a limit
158     if let Some(max_size) = max_size &&
159         // And the file is too large
160         {
161         let metadata = fs::metadata(entry_path).wrap_err(format!(
162             "Couldn't read metadata of file {}",
163             entry_path.display()
164         ))?;
165         metadata.len() > max_size
166     }
167     {
168         // So we skip the file
169         debug!("Skipping {} due to max-size", entry_path.display());
170         print!("{CLEAR_LINE}Skipping {}...", entry_path.display());
171         return Ok(Value::Null);
172     }
173
174     // Otherwise, there is no limit so we read the file
175     debug!("Reading {}...", entry_path.display());
176     print!("{CLEAR_LINE}Reading {}...", entry_path.display());

```

```

177
178     let bytes =
179         fs::read(entry_path).wrap_err(format!("Couldn't read file {}",
180             entry_path.display()))?;
181
182     // Try to parse it to UTF-8
183     let content = match String::from_utf8(bytes) {
184         Ok(s) => Value::Text(s),
185         // Skip file is not UTF-8 and text_only is active
186         Err(e) if !text_only => Value::Bytes(e.into_bytes()),
187         _ => {
188             debug!("Excluded {} due to text-only", entry_path.display());
189             Value::Null
190         }
191     };
192
193     // Include file path as well if we want that
194     Ok(if with_file_path {
195         Value::Array(vec![
196             Value::Text(entry_path.to_string_lossy().to_string()),
197             content,
198         ])
199     } else {
200         content
201     })
202
203 /// Parse either a plain number or a human-readable byte size.
204 fn parse_byte_size(s: &str) -> std::result::Result<u64, String> {
205     let s = s.trim();
206
207     // Try as plain integer first
208     if let Ok(n) = s.parse::<u64>() {
209         return Ok(n);
210     }
211
212     // Try as human-readable byte size
213     let pos = s.find(|c: char| !c.is_ascii_digit()).unwrap_or(s.len());
214     let number_str = &s[..pos];
215     let unit = &s[pos..];
216
217     let number: u64 = number_str
218         .parse()
219         .map_err(|_| format!("'{number_str}' is not a valid number"))?;
220
221     let factor = match unit.trim().to_ascii_lowercase().as_str() {

```

```
222     "k" | "kb" => 1_000,
223     "m" | "mb" => 1_000_000,
224     "g" | "gb" => 1_000_000_000,
225     "ki" | "kib" => 1 << 10,
226     "mi" | "mib" => 1 << 20,
227     "gi" | "gib" => 1 << 30,
228     _ => return Err(format!("{}",unit)' is not a valid unit')),
229 };
230
231 let bytes = number * factor;
232 Ok(bytes)
233 }
```